

实验 3 特权级与系统调用

截止时间：2026 年 5 月 20 日 23:59

提交要求：

- 正常提交→OBE 上“实验 3”；
- 截止后覆盖提交→OBE 上新开条目（待 OBE 上创建）；

实验报告编制要求：

- 文件名：学号姓名.pdf ←注意
 - ◆ 必须使用 PDF 格式，不能是 doc、docx、odt、md 等格式
- 报告标题：体现出“实验 3”等信息
- 报告作者：学号 姓名 ←注意
- 不同题目之间有明显的间隔，要能够容易地区分不同题目对应的实验内容
- 在报告开篇说明新增的文件和修改过的原有代码文件 ←注意
- 实验报告不做任何其他版式上的要求

提交要求：

- 提交完整可编译运行的源代码（提交前在 compile 目录执行 make clean 清楚二进制文件以减少体积）
 - ◆ 所有代码在提交前必须保证能在实验 2 所提及的服务器上编译通过
- 实验报告与源码目录一起压缩，压缩包必须使用 Linux 常用的压缩包格式.tar.gz，源码不用额外打包，源码目录与实验报告一起打包。
 - ◆ 假设源码目录包 Xinu-qemu-2026 和实验报告 123456 学生.pdf 在同一个目录
 - ◆ 在该目录中执行：tar czvf 学号.tar.gz .
 - ◆ 压缩包文件名为学号，上述命令最后有一个“.”

代码编写要求：

- 函数命名规则
 - ◆ 所有新增的内核态函数均以“k 学号_”作为前缀，例：学号为 123456 的同学实现了一个内核态 printf 函数，则该函数的函数名应为

“k123456_printf”

- ◆ 所有新增的**用户态**函数均以“u 学号_”作为前缀，例：学号为 123456 的同学实现了一个用户态 printf 函数，则该函数的函数名应为“u123456_printf”
- ◆ 在本实验手册内提及的新函数均应按照上述要求命名
- ◆ 此处所指的函数包括 C 语言函数和汇编代码函数
- **全局变量、结构体命名规则**
 - ◆ 所有新增的全局变量和结构体的命名也遵循函数命名规则
 - ◆ 局部变量不用遵循上述规则
- 实验可能用到的头文件 **tss.h** 已经上传到 OBE 的课件栏目 **←注意**
- 所有新增的其它结构体（如果需要）、全局变量声明（如果需要）、新的函数声明（如果需要），放到 include/**Lab3.h** 中，该头文件可在 xinu.h 中引用 **←注意**
- 所有新增的功能函数，放入 system 目录下以“Lab3_”为前缀名的源文件中；如果所有函数都在同一个源文件中，则该文件名称可为 Lab3.c；也可按照 Xinu 的习惯以函数本身的功能名称参与文件名，如 Lab3_printf.c **←注意**
- **对原有代码的修改**：单行改动，应以修改行前一行或当前行尾的注释“// Lab3 学号”标记；多行修改，应以首行行前“/*Lab3 学号: Begin*/”和末行行后“/*Lab3 学号: End*/”标记
- 在 shell 目录下新建一个文件 xsh_lab3.c，其中包含一个 shell 命令函数“xsh_lab3”，参数类型请参考其他 shell 命令
- 在 shell/shell.c 的 cmdtab 数组中，添加{“lab3”, FALSE, xsh_lab3}，注意保证语法正确（后续可在 Xinu 的 xsh 中通过 lab3 命令来测试所有题目）

本次实验**要求提交代码**，也需提交实验报告。实验报告应说明对代码的修改（新增、修改等），以及**丰富的截图和配套的文字说明**（不能只有截图而无文字说明，或只有文字而无截图）。

1. 实验目的

希望学生能掌握内核态与用户态之间控制转移的实现，将 Xinu 转化为一个更为现代化的操作系统内核。

2. 实验要求

2.1 实现用户态 shell 命令 xsh_lab3，应完成如下要求：

- a xsh_lab3 命令运行于用户态（应通过调试等方式展示该函数的指令运行于用户态）
- b xsh_lab3 可以创建另一个用户态的进程 uptest(int32, int32)，uptest 进程工作流程如下所示，但应注意创建用户态进程的功能应是一般化的，而不是只能针对这里的 uptest（uptest 函数实现在 xsh_lab3.c 中）

输出进程 ID 和第 1 个参数，如 `printf("proc=%d: a=%d", getpid(), a)`
睡眠一段时间（不少于 1 个时间片）
输出进程 ID 和第 2 个参数，如 `printf("proc=%d: b=%d", getpid(), b)`

- c xsh_lab3 中至少以不同参数创建并启动 uptest 进程 3 次
- d xsh_lab3 返回前输出学号和姓名（如果中文输出乱码，可使用拼音；具体如何实现输出，不做任何限制，但应在实验报告中明确说明）
- e xsh_lab3 和 uptest 均以 return 正常结束（不能以无限循环或是绕过 return 的函数调用结束）
- f 调试状态下，能查看到用户态进程的正确的进程名称；或者直接在用户进程中输出进程名称。以截图方式呈现在实验报告中
- g 在非调试状态下，shell 中执行 lab3 命令能够正常结束

2.2 （扩展要求，不强制）将 main、shell 和至少 echo、ps、help 这三个命令变成用户态进程

2.3 其他说明

- a 所有用户态进程的优先级为 INITPRIO (20)，内核态栈为 4KB，用户态栈为 8KB
- b 由于 Xinu 无法有效区分开内核态代码/数据和用户态代码/数据，可以在进入用户态进程之后认为所有全局变量都是用户态可访问的，但在本实

验中，用户态最多只允许访问 `xsh_lab3.c` 中定义的全局变量和 Xinu 的进程表项（用于 2.1.f 中输出进程名）

- c 创建用户态进程的函数调用与原来创建进程的函数调用 `create` 保持一样的参数列表（即使优先级和栈大小这两个参数可能没有实际作用），但可以是另外实现的函数（如 `create_user_proc` 专门用于创建用户态进程，而不修改原 `create` 的代码）。应在实验报告中明确说明创建用户态进程的函数以及工作流程（与原有 `create` 中类似的操作可以简略）。也可以选择修改 `create` 函数，使其既能创建用户态进程，也能创建内核态进程。
- d 建议实现顺序：①首先使一个没有任何具体功能的特定函数能进入用户态；②然后在该函数中添加循环，使其能正确的通过时间片轮转实现调度；③添加输出功能的系统调用相关实现；④添加或修改创建并启动一般化用户态进程的功能；⑤添加用户态进程正常退出的功能，并去除用户态进程函数中的循环。各位同学可根据实际情况自行决策如何完成。

3. 思维扩展

（不要求在实验报告中作答）

- 3.1 查阅资料，理解如何通过调用门从用户态进入内核态，并返回到用户态，并对比与课堂所介绍及本实验所实现的方式相比，有何异同或优劣；
- 3.2 通过调试或添加代码，查看进程切换前后 TSS 中的内容（如 CS、ESP、EIP、ESP0 等）是否发生变化。
- 3.3 通过查阅资料、阅读代码、调试运行等手段，理解 OpenHarmony 的 LiteOS-A 中用户态 `Init` 进程的完整创建流程，在什么时间点实际进入了用户态。